

**DRIVER DROWSINESS DETECTION
PROJECT REPORT**

Submitted by

PERAMBUDURI AISHWARYA (B192045)

PRERANA REDDY MALE (B192082)

GANDAM LAHARI (B192039)

Of

Bachelor of Technology

Under the guidance of

Mrs. G. Srujana

Asst. Prof. CSE, RGUKT BASAR



**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING
RAJIV GANDHI UNIVERSITY OF KNOWLEDGE
TECHNOLOGIES
BASAR, NIRAML (DIST),
TELANGANA – 504107**

DRIVER DROWSINESS DETECTION

Project Report submitted to
Rajiv Gandhi University of Knowledge Technologies, Basar
for the partial fulfillment of the requirements
for the award of the degree of
Bachelor of Technology

in

Computer Science and Engineering

by

PERAMBUDURI AISHWARYA (B192045)

PRERANA REDDY MALE (B192082)

GANDAM LAHARI (B192039)

Under the guidance of

Mrs. G. Srujana

Asst. Prof. CSE, RGUKT BASAR



**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING
RAJIV GANDHI UNIVERSITY OF KNOWLEDGE
TECHNOLOGIES**

MAY, 2025



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES,
BASAR
CERTIFICATE**

This is to certify that the Project Report entitled '**DRIVER DROWSINESS DETECTION**' submitted by **Perambuduri Aishwarya - B192045, Prerana Reddy Male – B192082, Gandam Lahari – B192039**, Department of Computer Science and Engineering, Rajiv Gandhi University Of Knowledge Technologies, Basar, for partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering; is a bonafide record of the work and investigations carried out by them under my supervision and guidance.

PROJECT SUPERVISOR:

Mr. P. Laxmi Narayana

Assistant Professor

HEAD OF THE DEPARTMENT:

Dr. B. Venkat Raman

Assistant Professor

PROJECT COORDINATOR:

Mrs. G. Srujana

Assistant Professor

EXTERNAL EXAMINER:



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RAJIV GANDHI UNIVERSITY OF KNOWLEDGE
TECHNOLOGIES,
BASAR
DECLARATION

We hereby declare that the work which is being presented in this project entitled, **"DRIVER DROWSINESS DETECTION"** submitted to **RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES, BASAR** in the partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING**, is an authentic record of our own work carried out under the supervision of **"Mrs. G. Srujana"**, Assistant Professor in Department of Computer Science And Engineering, RGUKT, Basar. The matter embodied in this project report has not been submitted by us for the award of any other degree.

Place:Basar

Date:09/05/25

P.AISHWARYA (B192045)

M.PRERANA REDDY(B192082)

G.LAHARI (B192039)

ACKNOWLEDGEMENT

We would like to express our deep gratitude to our project guide **Mrs.G.SRUJANA**, Assistant Professor, Department of Computer Science and Engineering, RGUKT BASAR, for her guidance with unsurpassed knowledge and immense encouragement.

We are grateful to **Dr. B. VENKAT RAMAN**, Head of the Department, Computer Science and Engineering, for providing us with the required facilities for the completion of the project work.

We are very much thankful to the **Director and Administration, RGUKT, BASAR** for their encouragement and cooperation to carry out this work.

We express our thanks to all **teaching faculty** of Department of CSE, whose suggestions during reviews helped us in accomplishment of our project.

We would like to thank **all non-teaching staff** of the Department of CSE, RGUKT BASAR for providing great assistance in accomplishment of our project.

We would like to thank our parents, friends, and classmates for their encouragement throughout our project period. At last but not the least, we thank everyone for supporting us directly or indirectly in completing this project successfully.

PROJECT STUDENTS:

P.AISHWARYA (B192045)

M.PRERANA REDDY (B192082)

G.LAHARI (B192039)

ABSTRACT

This project is a real-time Driver Drowsiness Detection System designed to prevent road accidents caused by driver fatigue. Using a webcam, the system continuously monitors the driver's facial features, particularly the eyes, to identify signs of drowsiness. The core functionality relies on facial landmark detection using Dlib and OpenCV, where the Eye Aspect Ratio (EAR) is calculated to determine if the driver's eyes remain closed for an extended period. If the EAR value drops below a certain threshold for a continuous number of frames, it is considered a sign of drowsiness.

Once drowsiness is detected, the system triggers alert mechanisms by playing preloaded audio files such as "alert-sound.mp3" to capture the driver's attention. The program also displays real-time feedback on the video screen with messages like "AWAKE," "BLINKING," or "DROWSY!" based on the driver's condition. This interactive interface ensures both monitoring and feedback are intuitive and effective.

The project demonstrates how computer vision and deep learning can be integrated to address real-world safety issues. It is lightweight, efficient, and ideal for integration into vehicles or driver assistance systems, ultimately contributing to safer driving environments.

TABLE OF CONTENTS

Certificate.....	III
Declaration.....	IV
Acknowledgement.....	V
Abstract.....	VI
Contents.....	VII

CHAPTER 1.....	1
-----------------------	----------

INTRODUCTION

1.1 Project Objective.....	1
1.2 Background.....	2
1.3 Problem Statement.....	3
1.4 Aims and Objectives.....	4
1.5 Scope of the Project.....	5
1.6 Flowchart.....	6

CHAPTER 2.....	7
-----------------------	----------

REVIEW OF LITERATURE

2.1 Existing System.....	7
2.2 Disadvantages.....	8
2.3 Proposed System.....	9
2.4 Advantages of Proposed System.....	10

CHAPTER 3.....	11
ALGORITHMS INVOLVED IN DRIVER DROWSINESS DETECTION	
Algorithms.....	11
CHAPTER 4.....	14
MODEL IMPLEMENTATION AND ANALYSIS	
4.1 Introduction.....	14
4.2 Design Requirements.....	15
CHAPTER 5.....	16
CODE IMPLEMENTATION	
5.1 Code Implementation.....	16
5.2 Output Images.....	21
CHAPTER 6.....	22
Conclusion and Future Works.....	22
References.....	23

CHAPTER 1

INTRODUCTION

1.1 Project Objective

The objective of the Driver Drowsiness Detection System is to enhance road safety by utilizing deep learning and computer vision techniques to detect driver drowsiness in real-time and issue timely alerts. The system uses facial landmark detection and the Eye Aspect Ratio (EAR) to monitor the driver's level of alertness. In addition, it integrates Mouth Aspect Ratio (MAR) to track signs of fatigue and detect if the driver is yawning, further improving the system's accuracy in identifying drowsiness.

The system processes video input through a camera to analyze the driver's facial features and track changes in eye and mouth movements. If the system detects prolonged eye closure or frequent yawning, indicative of fatigue, it triggers an audio alert to warn the driver, allowing them to take necessary precautions.

This project leverages deep learning models alongside Python libraries such as OpenCV, Dlib for facial feature recognition and tracking. The system runs locally on a desktop environment, making it both efficient and cost-effective compared to commercial solutions, which rely on cloud processing or expensive hardware.

By combining deep learning with facial and mouth aspect ratios, this system offers an accurate, real-time solution for preventing accidents caused by drowsy driving, demonstrating the practical application in improving road safety.

1.2 Background

Drowsy driving is a major contributor to road accidents worldwide, with fatigue impairing a driver's attention, reaction time, and decision-making abilities. Studies show that driving while drowsy can be as dangerous as driving under the influence of alcohol. As a result, there is a growing need for solutions that can monitor and mitigate the risks of drowsy driving.

Advancements in computer vision and deep learning have enabled the development of systems that can detect signs of driver fatigue using facial landmarks, eye movements, and other physical indicators. By analyzing the Eye Aspect Ratio (EAR), which measures eye openness, and the Mouth Aspect Ratio (MAR), which detects yawning, these systems can accurately monitor driver alertness in real-time. When drowsiness is detected, an alert can be triggered to warn the driver, helping to prevent accidents caused by fatigue.

Traditional drowsiness detection systems often rely on expensive hardware or cloud-based processing, which can be cost-prohibitive and may introduce delays in real-time applications. However, by utilizing open-source libraries like OpenCV, Dlib, and deep learning models, this system provides an efficient, cost-effective, and real-time solution for detecting driver drowsiness without the need for specialized hardware.

This project focuses on creating an accessible and effective drowsiness detection system, contributing to safer driving environments and reducing the risk of accidents due to fatigue.

1.3 Problem Statement

Driver drowsiness is a critical factor contributing to road accidents, posing significant risks to human lives and property. The need for an efficient, real-time system to detect and alert drowsy drivers is paramount to enhancing road safety. This project aims to develop a driver drowsiness detection system leveraging deep learning techniques, utilizing Dlib and OpenCV libraries. The system will analyze facial features, eye movements from video feed captured via a camera to identify signs of drowsiness, such as prolonged eye closure, frequent yawning.

Using Dlib's facial landmark detection, the system will extract key facial points to monitor eye aspect ratio (EAR) and mouth opening patterns. OpenCV will process real-time video frames, enabling robust image preprocessing and feature extraction. A deep learning model, trained on labeled datasets of drowsy and alert drivers, will classify driver states based on these features. The system will issue timely alerts (e.g., audio or visual) upon detecting drowsiness, ensuring proactive intervention. Challenges include varying lighting conditions, diverse facial structures, and real-time processing constraints, which the system must address for reliable performance across different driving scenarios.

1.4 Aims & Objectives

Aims:

1. To develop a real-time Driver Drowsiness Detection System using deep learning and computer vision techniques.
2. To enhance road safety by alerting drivers when signs of drowsiness or fatigue are detected.
3. To create a cost-effective, locally running solution without reliance on cloud services or expensive hardware.
4. To demonstrate the practical application of DL and open-source libraries in safety-critical environments.

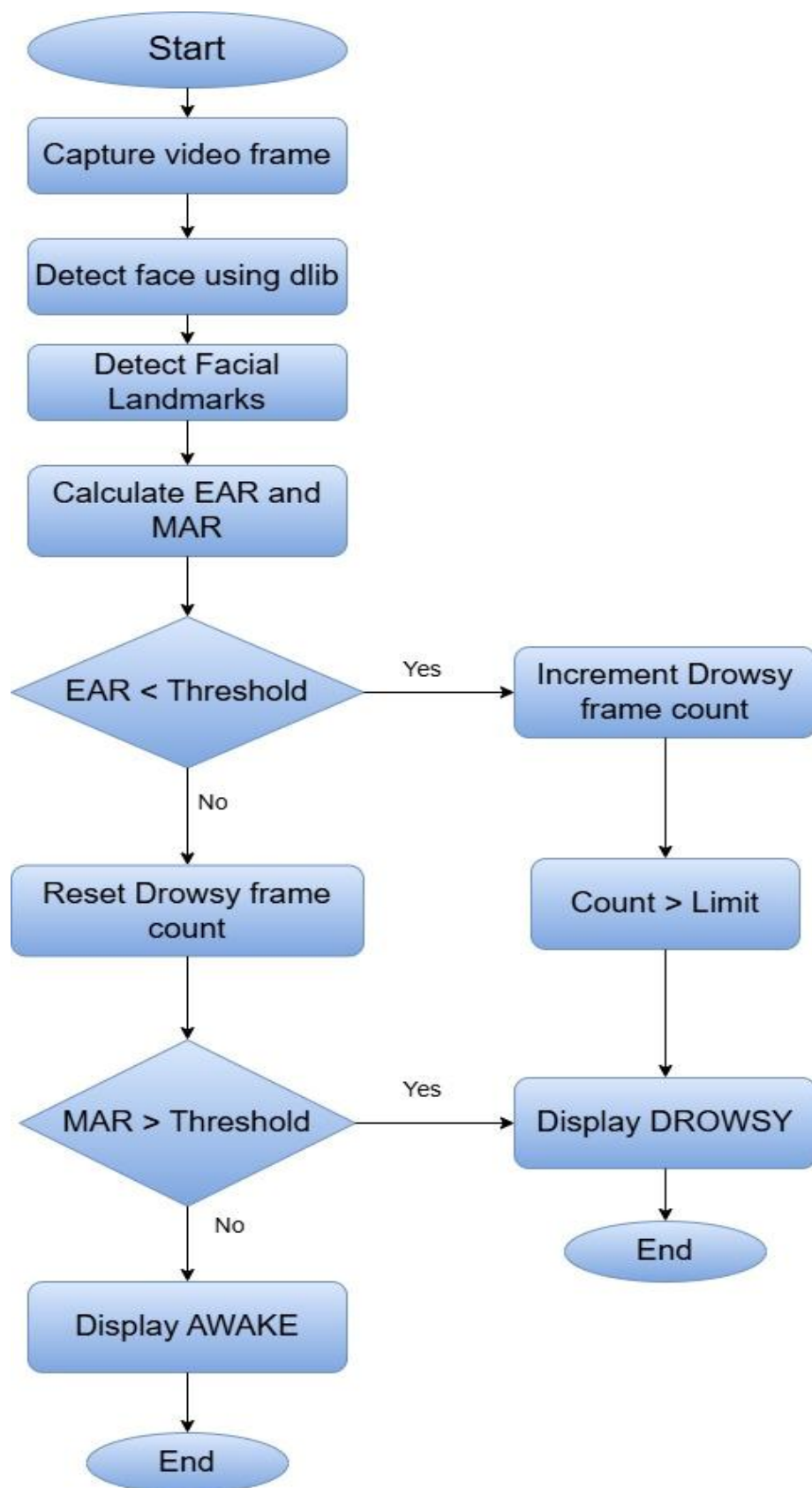
Objectives:

1. To train or integrate a deep learning model capable of identifying drowsy states.
2. To design a system that triggers audio alerts when drowsiness is detected.
3. To ensure the system runs efficiently in real-time on standard desktop environments.
4. The solution will maintain high performance with minimal resource consumption, ensuring smooth and responsive monitoring.
5. To test the system under different lighting and head movement conditions. This will improve the system's robustness and reduce false detections, making it reliable in various real-world scenarios.
6. To provide a user-friendly and open-source solution for further research and development.
7. The project will be easy to use, modify, and extend, encouraging further innovation in driver safety technologies.

1.5 Scope of the Project

This project aims to build a desktop-based Driver Drowsiness Detection System that monitors a driver's alertness through facial behavior analysis. It utilizes deep learning and computer vision to detect early signs of fatigue, such as frequent blinking, prolonged eye closure, and yawning. By calculating key metrics like Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR), the system identifies abnormal facial patterns associated with drowsiness. The solution is designed to function in real-time using only a standard webcam and can operate without dependence on external hardware. It will generate an immediate audio alert when drowsiness is detected, prompting the driver to take corrective action. The project is lightweight, open-source, and intended for educational, research, and safety applications. Additionally, it lays a strong foundation for future enhancements such as head pose estimation, mobile deployment, or integration into advanced driver-assistance systems (ADAS) for broader transportation safety.

1.6 Flow Chart



CHAPTER-2

REVIEW OF LITERATURE

2.1 Existing System

Current systems for detecting driver drowsiness are primarily implemented in high-end vehicles through Advanced Driver Assistance Systems (ADAS), which integrate multiple sensors and technologies. These existing solutions often utilize a combination of infrared cameras, steering wheel sensors, lane deviation monitoring, and physiological measures like heart rate or eye-tracking sensors. For example, systems like Mercedes-Benz Attention Assist and BMW Driver Attention Camera monitor driver behavior, alerting them if their driving becomes erratic or if signs of drowsiness are detected. Despite their effectiveness in detecting drowsiness, existing systems come with a variety of limitations. Many systems require expensive hardware, such as specialized infrared cameras or sensors, which make them cost-prohibitive for widespread consumer adoption. Additionally, some rely on cloud-based processing, which introduces latency and may not offer real-time alerts necessary for effective drowsiness detection. These systems also generally depend on proprietary technology, making them difficult to adapt for open-source development or broader integration.

In contrast, the proposed solution aims to address these gaps by providing an efficient, real-time, open-source, and cost-effective alternative that can run on standard desktop computers using easily accessible webcams and open-source libraries, making it more scalable and widely applicable.

2.2 Disadvantages of the Existing Systems

- ***High Cost of Hardware:***

Many existing systems use specialized hardware like infrared cameras, which can be expensive and not affordable for widespread consumer use. This makes the technology inaccessible for budget-conscious individuals or smaller vehicle manufacturers.

- ***Dependency on Proprietary Technology:***

Most systems rely on proprietary solutions that cannot be easily adapted, modified, or integrated with third-party applications. This limits customization and scalability.

- ***Latency Due to Cloud Processing:***

Some systems process data on the cloud, introducing latency and delays in detecting drowsiness. Real-time detection is crucial for safety, and any delay can reduce the effectiveness of the system.

- ***Limited Accessibility and Flexibility:***

Many existing systems are integrated into high-end vehicles, making them inaccessible to drivers of older or more budget-friendly cars. This reduces the system's impact on road safety across a broader demographic.

- ***Complicated Setup:***

Systems often require complex setups or calibration, making them difficult for non-technical users to implement or maintain.

2.3 Proposed System

The proposed system is a real-time driver drowsiness detection model that uses deep learning and computer vision techniques to enhance road safety. It leverages OpenCV, dlib, and Matplotlib to monitor the driver's face through a webcam. The system detects key facial landmarks using dlib's 68-point predictor, focusing on the eyes and mouth to assess signs of drowsiness.

Two key metrics are used: Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR). EAR helps identify prolonged eye closure—a strong indicator of drowsiness—by calculating the distance between specific eye landmarks. Similarly, MAR monitors yawning behavior, another common sign of fatigue, by measuring the openness of the mouth using mouth landmarks. When EAR drops below a threshold or MAR rises above a certain limit for a set time period, the system classifies the driver as drowsy.

Upon detection, an audio alarm is triggered to alert the driver and reduce the risk of accidents. The system is non-intrusive, cost-effective, and suitable for real-time deployment in vehicles. By combining both eye and mouth behavior analysis, the model offers more accurate and reliable detection. It can be further enhanced with infrared cameras for low-light use or integrated with vehicle control systems for automation.

Additionally, the system logs drowsiness events, which can be used for further analysis or reporting. It is designed to run efficiently on low-power devices, making it suitable for embedded automotive applications. This robust, multi-feature approach significantly improves detection accuracy over single-metric systems.

2.4 Advantages of the Proposed System

- ***Real-time-Monitoring***

The system continuously analyzes the driver's eye and mouth movements through a live video feed. This enables immediate detection of drowsiness symptoms, reducing response time and increasing safety.

- ***Non-Invasive***

It does not require any wearable device or physical contact with the driver. A standard webcam is sufficient, making it user-friendly and comfortable for long-term use.

- ***Lightweight-Efficient***

The algorithm is optimized to run on standard computing hardware with minimal resource usage. It can also be deployed on embedded systems used in automotive applications.

- ***Easy-Integration***

The system can be easily integrated into vehicles or driver assistance platforms. It enhances existing safety features without requiring extensive hardware modifications.

- ***Customizable-Thresholds***

EAR and MAR threshold values can be adjusted based on user behavior, age, or driving conditions. This flexibility ensures better accuracy across diverse driver profiles.

- ***Scalable***

It can be deployed across various transport systems, including taxis, buses, trucks, and personal vehicles. The system supports fleet-wide implementation for improved road safety.

CHAPTER-3

ALGORITHMS INVOLVED IN DRIVER DROWSINESS DETECTION

3.1 Haar Cascades (Deep Learning Approach)

Algorithm:

Uses a cascade of classifiers to detect objects. While traditionally a machine learning approach, deep learning can enhance Haar cascades by refining the feature extraction and classifier stages.

How it works:

1. Feature Extraction (Deep Learning): Instead of manually computing Haar-like features, deep learning methods use Convolutional Neural Networks (CNNs) to automatically learn discriminative features from images.
2. Training the Model: The CNN is trained to detect the object (like a face) by learning from labeled data (positive and negative examples). This approach allows the network to automatically adjust its feature extraction process during training.
3. Cascade Network: The CNN can be used in a cascading fashion, where multiple layers of networks progressively filter out non-object regions and refine object detection.
4. Detection: The trained deep learning model is applied to the image, where it processes different regions of the image to detect objects like faces.

In-our-code:

This is typically implemented using the OpenCV CascadeClassifier for object detection, which leverages pre-trained Haar cascade classifiers for detecting faces, eyes, etc.

3.2 Dlib (Deep Learning Approach)

Algorithm: Uses a pretrained ensemble of regression trees to detect 68 facial landmarks on the face.

How it works:

- **Face Detection:** dlib first detects faces using a **Histogram of Oriented Gradients (HOG)** feature extractor combined with a **linear SVM** classifier.
- **Landmark Prediction:** For each detected face, it uses a shape predictor (`shape_predictor_68_face_landmarks.dat`) trained on labeled facial data. This model uses **regression trees** to predict 68 facial points such as eyes, eyebrows, nose, mouth, and jawline.
- The algorithm works in stages, refining the position of each landmark over multiple passes to increase accuracy.

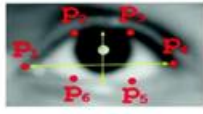
In our code:

This is handled using `dlib.shape_predictor("shape_predictor_68_landmarks.dat")` for extracting facial landmarks. These landmarks are then compute EAR and MAR to detect signs of drowsiness.

3.3 EAR&MAR

The Eye Aspect Ratio (EAR) is a widely used metric to measure eye openness in applications like facial expression analysis, eye tracking, and driver drowsiness detection. EAR is calculated using six eye landmarks: p1 to p6. The vertical eye distances (between p2–p6 and p3–p5) are divided by the horizontal eye distance (between p1 and p4). When the eyes are open, vertical distances are significant, resulting in a higher EAR value—typically around 0.3. As the eyes begin to close, these distances reduce, leading to a lower EAR. A threshold value of 0.3 is often used to detect partial or full eye closure, signaling drowsiness or fatigue. This conservative threshold ensures fewer missed detections while maintaining accuracy. Continuous monitoring of EAR allows real-time detection of eye closure, making it a reliable

method for alerting drivers and preventing fatigue-related accidents. EAR is efficient, easy to compute, and effective in real-time safety applications.

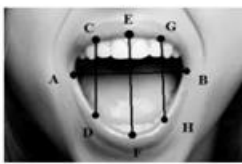


$$EAR = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2\|p_1 - p_4\|}$$

Figure: Eye Aspect Ratio Formula

The Mouth Aspect Ratio (MAR) measures mouth openness and is used in emotion and facial expression detection. It is calculated by dividing the vertical lip distance (EF) by the horizontal mouth width (AB). An open mouth results in a higher MAR, indicating expressions like surprise or happiness, while a closed mouth reduces MAR, suggesting emotions like sadness or fatigue. When MAR is below a threshold, it may indicate partial or full mouth closure, a potential sign of drowsiness. MAR is often used alongside the Eye Aspect Ratio (EAR) for reliable drowsiness detection.

After training a model with facial data, OpenCV is used to capture real-time frames. These undergo the same preprocessing steps—face detection, ROI extraction, resizing, and array conversion—before prediction. EAR and MAR are calculated from each frame. If either falls below its threshold, an alert stage is triggered, playing a loud beep or voice signal to warn the driver and promote corrective action, enhancing road safety.



$$MAR = \frac{|EF|}{|AB|}$$

Figure: Mouth Aspect Ratio Formula

CHAPTER 4

MODEL IMPLEMENTATION AND ANALYSIS

4.1 Introduction

The Driver Drowsiness Detection System is a real-time safety application developed to monitor driver alertness using a webcam and computer vision techniques. The system begins by continuously capturing video frames from the webcam. Within each frame, it detects the driver's face using models such as Dlib or Haar cascades. Once the face is detected, the system identifies 68 facial landmarks to pinpoint key regions, particularly the eyes and mouth, which are essential indicators of drowsiness and fatigue. These facial landmarks serve as the basis for calculating eye and mouth aspect ratios.

The Eye Aspect Ratio (EAR) is calculated from the distance between the vertical and horizontal eye landmarks. A sudden or prolonged drop in EAR below a certain threshold signifies that the driver's eyes are either closed or blinking frequently. Similarly, the Mouth Aspect Ratio (MAR) is derived from the width and height of the mouth landmarks. A high MAR sustained over several frames suggests yawning, another strong indicator of tiredness. The system maintains counters to track how long these conditions persist. If either the EAR or MAR stays in the warning range for too long, it interprets this as a sign of drowsiness.

Upon detecting signs of drowsiness, the system activates alerts to warn the driver. These alerts include visual messages displayed on the screen such as "DROWSY" or "YAWNING", and audio notifications that play sounds like beeps or verbal prompts from preloaded MP3 files. The combination of visual and audio alerts ensures that the driver receives immediate feedback to regain attention. This approach blends artificial intelligence, computer vision, and human-computer interaction to offer a practical and effective safety solution for reducing fatigue-related road accidents.

4.2 Design Requirements:

Camera-Enabled Device: A webcam or camera module is essential for continuously capturing real-time facial video input of the driver.

Python Environment with Required Libraries: The system must be run in a Python environment with essential libraries such as OpenCV for video processing, Dlib for facial landmark detection, Scipy and Numpy for numerical operations, and optionally playsound or pygame for playing alert sounds.

Facial Landmark Detection Models: Pre-trained models such as `shape_predictor_68_face_landmarks.dat` are required to accurately locate and monitor eye regions for EAR (Eye Aspect Ratio) computation.

Audio Alert Files: Preloaded audio files like `alert-sound.mp3`, `focus.mp3`, and `take_a_break.mp3` must be available in the working directory to provide timely auditory feedback.

Sufficient Lighting Conditions: For reliable detection, the environment must have adequate lighting to ensure the camera can accurately capture facial features.

Compatible System Hardware: The system should have sufficient CPU/GPU resources to perform real-time video processing and landmark detection without lag.

User Interface (Optional): A basic GUI or real-time display window using OpenCV to provide visual status updates like "AWAKE", "DROWSY", or "BLINKING".

CHAPTER-5

CODE IMPLEMENTATION

5.1 CODE IMPLEMENTATION

```
def yawn(mouth):

    return ((euclideanDist(mouth[2], mouth[10])+euclideanDist(mouth[4],
mouth[8]))/(2*euclideanDist(mouth[0], mouth[6])))

def getFaceDirection(shape, size):

    image_points = np.array([

        shape[33], # Nose tip

        shape[8], # Chin

        shape[45], # Left eye left corner

        shape[36], # Right eye right corne

        shape[54], # Left Mouth corner

        shape[48] # Right mouth corner

    ], dtype="double")

    # Camera internal

    focal_length = size[1]

    center = (size[1]/2, size[0]/2)

    camera_matrix = np.array(

        [[focal_length, 0, center[0]],

        [0, focal_length, center[1]],

        [0, 0, 1]], dtype = "double"

    )

    dist_coeffs = np.zeros((4,1)) # Assuming no lens distortion

    (success, rotation_vector, translation_vector) = cv2.solvePnP(model_points, image_points,
camera_matrix, dist_coeffs, flags=cv2.SOLVEPNP_ITERATIVE)
```



```

    return(translation_vector[1][0])

def euclideanDist(a, b):

    return (math.sqrt(math.pow(a[0]-b[0], 2)+math.pow(a[1]-b[1], 2)))

#EAR -> Eye Aspect ratio

def ear(eye):

    return ((euclideanDist(eye[1], eye[5])+euclideanDist(eye[2], eye[4]))/(2*euclideanDist(eye[0],
eye[3])))

def writeEyes(a, b, img):

    y1 = max(a[1][1], a[2][1])

    y2 = min(a[4][1], a[5][1])

    x1 = a[0][0]

    x2 = a[3][0]

    cv2.imwrite('left-eye.jpg', img[y1:y2, x1:x2])

    y1 = max(b[1][1], b[2][1])

    y2 = min(b[4][1], b[5][1])

    x1 = b[0][0]

    x2 = b[3][0]

    cv2.imwrite('right-eye.jpg', img[y1:y2, x1:x2])

# open_avg = train.getAvg()

# close_avg = train.getAvg()

alert = vlc.MediaPlayer('focus.mp3')

frame_thresh_1 = 15

frame_thresh_2 = 10

frame_thresh_3 = 5

close_thresh = 0.3#(close_avg+open_avg)/2.0

flag = 0

```

```

yawn_countdown = 0

map_counter = 0

map_flag = 1

capture = cv2.VideoCapture(0)

avgEAR = 0

detector = dlib.get_frontal_face_detector()

predictor = dlib.shape_predictor('shape_predictor_68_face_landmarks.dat')

(leStart, leEnd) = face_utils.FACIAL_LANDMARKS_IDXS["left_eye"]

(reStart, reEnd) = face_utils.FACIAL_LANDMARKS_IDXS["right_eye"]

(mStart, mEnd) = face_utils.FACIAL_LANDMARKS_IDXS["mouth"]

while(True):

    ret, frame = capture.read()

    size = frame.shape

    gray = frame

    rects = detector(gray, 0)

    if(len(rects)):

        shape = face_utils.shape_to_np(predictor(gray, rects[0]))

        leftEye = shape[leStart:leEnd]

        rightEye = shape[reStart:reEnd]

        leftEyeHull = cv2.convexHull(leftEye)

        rightEyeHull = cv2.convexHull(rightEye)

        # print("Mouth Open Ratio", yawn(shape[mStart:mEnd]))

        leftEAR = ear(leftEye) #Get the left eye aspect ratio

        rightEAR = ear(rightEye) #Get the right eye aspect ratio

        avgEAR = (leftEAR+rightEAR)/2.0

        eyeContourColor = (255, 255, 255)

```

```

if(yawn(shape[mStart:mEnd])>0.6):

    cv2.putText(gray, "Yawn Detected", (50,50), cv2.FONT_HERSHEY_COMPLEX,
1,(0,255,127),2)

    yawn_countdown=1

if(avgEAR<close_thresh):

    flag+=1

    eyeContourColor = (0,255,255)

    print(flag)

if(yawn_countdown and flag>=frame_thresh_3):

    eyeContourColor = (147, 20, 255)

    cv2.putText(gray, "Drowsy after yawn", (50,50), cv2.FONT_HERSHEY_COMPLEX,
1,(0,255,127),2)

    alert.play()

    if(map_flag):

        map_flag = 0

        map_counter+=1

    elif(flag>=frame_thresh_2 and getFaceDirection(shape, size)<0):

        eyeContourColor = (255, 0, 0)

        cv2.putText(gray, "Drowsy (Body Posture)", (50,50), cv2.FONT_HERSHEY_COMPLEX,
1,(0,255,127),2)

        alert.play()

        if(map_flag):

            map_flag = 0

            map_counter+=1

    elif(flag>=frame_thresh_1):

        eyeContourColor = (0, 0, 255)

```

```

        cv2.putText(gray, "Drowsy (Normal)", (50,50), cv2.FONT_HERSHEY_COMPLEX,
1,(0,255,127),2)

        alert.play()

        if(map_flag):

            map_flag = 0

            map_counter+=1

        elif(avgEAR>close_thresh and flag):

            print("Flag reseted to 0")

            alert.stop()

            yawn_countdown=0

            map_flag=1

            flag=0

        if(map_counter>=3):

            map_flag=1

            map_counter=0

            vlc.MediaPlayer('take_a_break.mp3').play()

            webbrowser.open("https://www.google.com/maps/search/hotels+or+motels+near+me")

        cv2.drawContours(gray, [leftEyeHull], -1, eyeContourColor, 2)

        cv2.drawContours(gray, [rightEyeHull], -1, eyeContourColor, 2)

        writeEyes(leftEye, rightEye, frame)

        if(avgEAR>close_thresh):

            alert.stop()

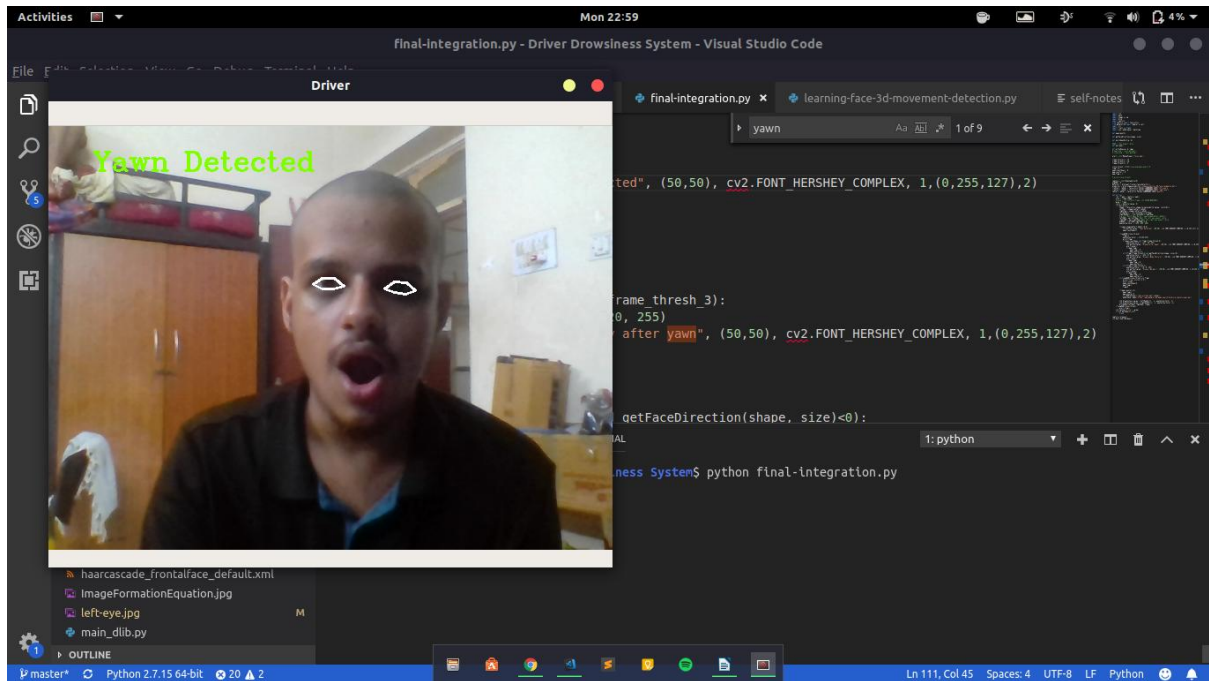
        cv2.imshow('Driver', gray)

capture.release()

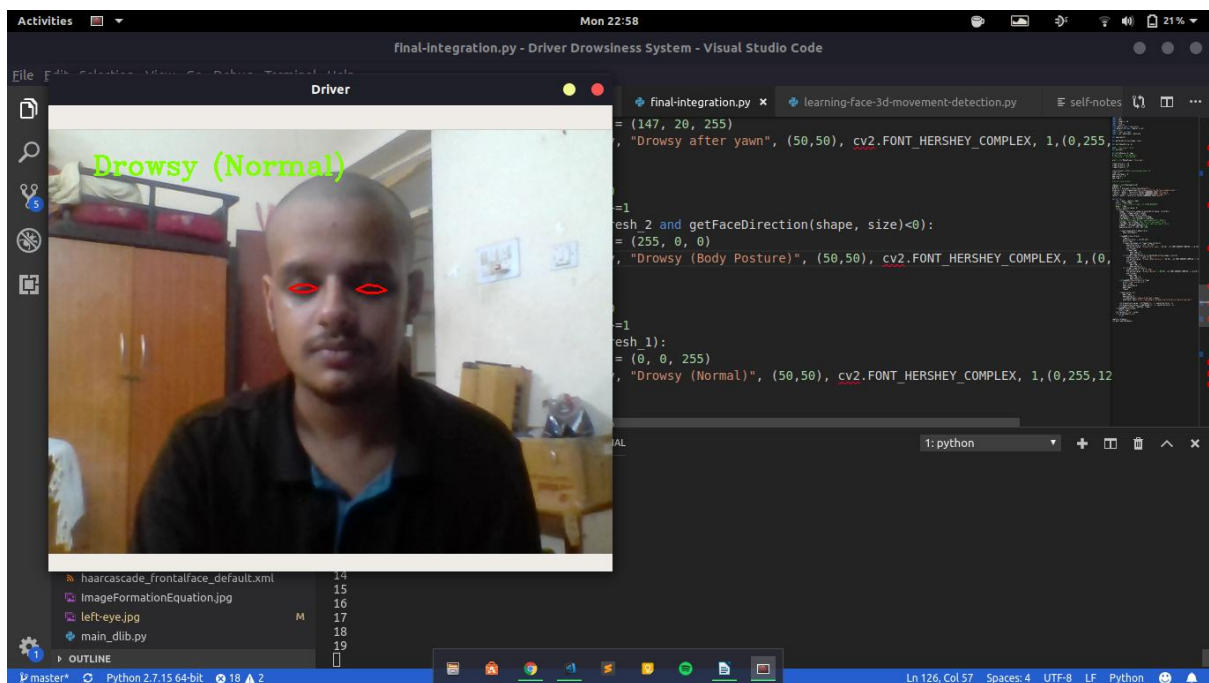
cv2.destroyAllWindows()

```

5.2 Output Images



5.2.1. Yawn Detection Image



5.2.2. Drowsy Detection Image

CHAPTER-6

CONCLUSION AND FUTURE WORKS

The Driver Drowsiness Detection System successfully demonstrates how computer vision and deep learning can be applied to enhance driver safety on roads. Using Python-based technologies like OpenCV and Dlib, the system continuously monitors a driver's eyes through a webcam feed and calculates the Eye Aspect Ratio (EAR) to determine whether the eyes are open or closed. By tracking this ratio in real-time, the system can distinguish between normal blinking and potential signs of drowsiness, triggering an audio alert if the driver appears to be dozing off. This solution provides a non-intrusive and efficient method of detecting fatigue and aims to reduce accidents caused by microsleep or inattention.

The implementation is designed to be lightweight and runs in real-time, making it suitable for integration into vehicle dashboards or mobile computing platforms. It relies on accurate facial landmark detection and sound logic to classify states like "Awake," "Blinking," or "Drowsy." Furthermore, audio alerts using predefined sounds ensure that the driver is warned in time to regain focus. The current system is a strong prototype that validates the concept of drowsiness detection through eye monitoring and sets the foundation for more sophisticated applications in automotive safety.

In the future, this project can be expanded to include features like head pose detection, yawn detection, and heart rate monitoring for more comprehensive fatigue assessment. The integration of deep learning models, such as convolutional neural networks, could improve detection accuracy under different lighting and camera angles. Additionally, deploying the system on embedded platforms like Raspberry Pi or integrating it into existing car systems with GPS and alert messaging capabilities would make it more practical for real-world use. These enhancements will contribute to a more intelligent and robust driver assistance system capable of significantly reducing road accidents.

REFERENCES

- Soukupová, T., & Čech, J. (2016). Real-Time Eye Blink Detection using Facial Landmarks. Center for Machine Perception, Czech Technical University, Prague.
- <https://vision.fe.uni-lj.si/cvww2016/proceedings/papers/05.pdf>
- Kazemi, V., & Sullivan, J. (2014). One millisecond face alignment with an ensemble of regression trees. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1867–1874.
- OpenCV Library – Open Source Computer Vision Library. <https://opencv.org/>
- Dlib C++ Library – A toolkit for machine learning and data analysis. <http://dlib.net/>
- Tereza Soukupova and Jan Cech, Facial Landmark Detection Using Dlib, Czech Technical University, Prague.
- Python Playsound Module – Simple library for playing sounds. <https://pypi.org/project/playsound/>
- Viola, P., & Jones, M. (2001). Rapid Object Detection using a Boosted Cascade of Simple Features. Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR).
- GeeksforGeeks – Eye Aspect Ratio (EAR) Based Drowsiness Detection Using Python.
- <https://www.geeksforgeeks.org/eye-blink-detection-using-opencv-python/>